B.TECH CCE REVISION · PART 3

Computer Organisation and Operating Systems

Covers the marksheet subjects Digital Systems, Digital System Design, Computer Organisation, Operating Systems, and the useful parts of Embedded Systems.

From logic gates to a CPU, the memory hierarchy, and what an operating system actually does: processes, threads, scheduling, virtual memory, concurrency, files, and how containers differ from virtual machines.

Isagi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

How to use this chapter

Operating systems were the subject flagged as intimidating. Most of that comes from the exam style: scheduling tables to fill in, page-replacement traces to compute by hand. The ideas themselves are few and intuitive, and they explain a lot of everyday engineering: why a server runs out of memory, why two requests corrupt each other's data, why containers start in a second and VMs in a minute, why "async" exists.

The chapter goes bottom-up: from switches to a CPU, then memory, then the operating system that shares all of it among many programs.

> **How to read it.** One section per sitting, then its lab in Section 11. No proofs, no hand-traced algorithms. The self-check questions in Section 12 are the finish line.

Contents

1. Bits, logic gates and digital circuits
2. From circuits to a CPU
3. The memory hierarchy
4. What an operating system is for
5. Processes and threads
6. Scheduling and context switching
7. Virtual memory
8. Concurrency: races, locks, deadlock, async
9. Files, I/O and system calls
10. Virtual machines and containers
11. Labs
12. Self-check questions and answers
13. Resources

1. Bits, logic gates and digital circuits

Everything a computer does is built from switches that are either on or off.

- A **transistor** is an electrically controlled switch. A modern phone chip has over 15 billion of them.
- Arrange a few transistors and you get a **logic gate**: AND (output 1 only if both inputs are 1), OR, NOT, and the universal ones, **NAND** and NOR. Every digital circuit can be built from NAND gates alone; that's the premise of Nand2Tetris and the game Turing Complete.
- **Binary** represents numbers in base 2. Eight bits make a byte (0–255). Negative numbers use **two's complement**, which lets the same adder circuit do subtraction. Text is numbers too (UTF-8 encodes every character, including Hindi and emoji, as 1–4 bytes).
- **Hexadecimal** is shorthand for binary: each hex digit is four bits (`0xFF` = 255). You see it in memory addresses, colours (`#0f766e`) and MAC addresses.

There are two kinds of circuits:

Kind	Output depends on	Examples
Combinational	Only the current inputs	Adder, multiplexer (a selector switch), decoder
Sequential	Current inputs *and* stored state	Flip-flops, registers, counters, memory

A **flip-flop** stores one bit. It's the basic unit of memory. A **clock** is a signal ticking billions of times a second; sequential circuits update their state only on a clock tick, which keeps everything in step. A 3 GHz clock ticks 3 billion times per second, so one tick is a third of a nanosecond, during which light travels only 10 cm.

Worked example: adding with gates

Adding two single bits A and B gives a sum bit (A XOR B) and a carry bit (A AND B). That's a **half adder**. Chain 64 **full adders** (which also take a carry-in) and you have the 64-bit adder at the heart of a CPU. The point isn't to memorise the circuit; it's that arithmetic is just gates wired in a pattern, and nothing more mysterious is going on.

2. From circuits to a CPU

A CPU is a loop: fetch an instruction, decode it, execute it, repeat, billions of times a second.

The main parts:

- **Registers**: a handful of very fast storage slots inside the CPU (x86-64 has 16 general-purpose ones). All arithmetic happens on registers.
- **ALU** (arithmetic logic unit): does add, subtract, compare, AND, OR and so on.
- **Control unit**: reads each instruction and sets the switches so data flows to the right places.
- **Program counter**: a register holding the address of the next instruction.

The instruction cycle: fetch the instruction at the program counter from memory → decode it → execute (maybe load from or store to memory) → advance the program counter (or jump, for an `if` or a loop) → repeat.

An **instruction set architecture (ISA)** is the vocabulary a CPU understands. **x86-64** (Intel, AMD) runs most PCs and servers. **ARM** runs every phone, Apple's M-series Macs and an increasing share of cloud servers (AWS Graviton), mainly because it delivers more work per watt. Your Python code is turned into these instructions by an interpreter or compiler.

Modern CPUs are much faster than the simple loop suggests because of:

- **Pipelining**: like an assembly line, different instructions are in different stages at once.
- **Multiple cores**: several independent CPUs on one chip. An 8-core laptop can genuinely run 8 things at the same instant.
- **Branch prediction and speculative execution**: the CPU guesses which way an `if` goes and runs ahead. (Guessing wrong and leaking what was guessed is what the Spectre attacks exploited.)
- **SIMD**: one instruction operating on many numbers at once. **GPUs** take this to the extreme: thousands of simple cores doing the same operation on different data, which is exactly what matrix multiplication in neural networks needs. That's why AI runs on GPUs.

Embedded systems are the same ideas in miniature: a **microcontroller** (an Arduino's ATmega, an ESP32) is a small CPU, memory and input/output pins on one chip, often running a single program with no full operating system, inside things like washing machines, paint-tinting machines and IoT sensors.

3. The memory hierarchy

Fast memory is small and expensive; big memory is slow and cheap. Computers stack them and keep the hot data near the top.

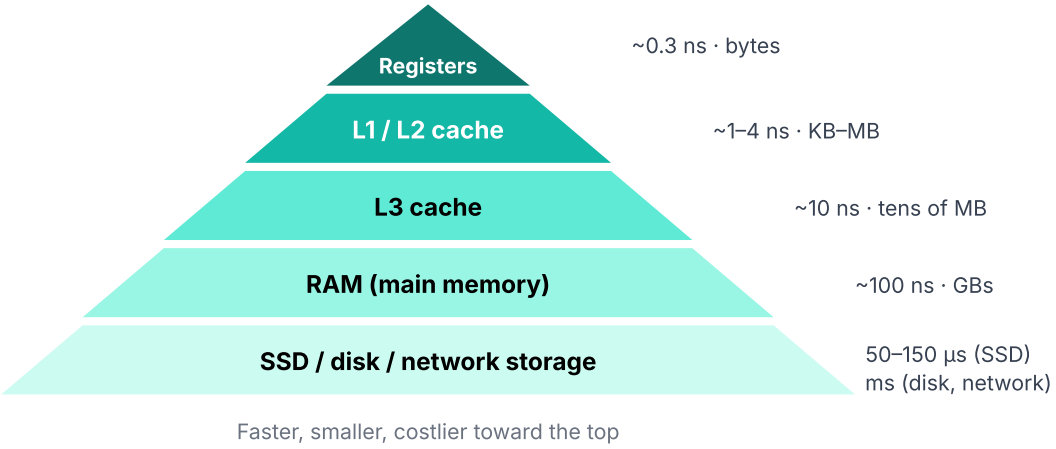


Figure 1. The memory hierarchy, with rough access times.

To feel the gaps, scale them to human time. If reading a register took 1 second, reading RAM would take about 5 minutes, reading an SSD a few days, and a round trip from India to a US server about 25 years.

Caches work because programs have **locality**: they reuse data they touched recently (*temporal*) and touch data near it (*spatial*). The CPU loads memory in 64-byte **cache lines**, so looping through an array in order is fast and jumping around randomly is slow, even though the code looks equally simple.

The same principle repeats at every level of software: a database keeps hot pages in RAM, a web app caches query results in Redis, a CDN caches files near users. **Caching is the memory hierarchy applied to systems**, and cache invalidation (knowing when the copy is stale) is the hard part everywhere.

Memory	Survives power off?	Notes
Registers, cache, RAM	No (volatile)	RAM is why a server "forgets" when it restarts
SSD (flash)	Yes	No moving parts; wears out after many writes
Hard disk	Yes	Spinning platters; slow random access, cheap per TB

4. What an operating system is for

The OS shares one machine's hardware safely among many programs, and gives each one a simpler machine to program against.

It does three jobs:

1. **Resource sharing**: decides which program gets the CPU when, how memory is divided, who uses the disk and network.
2. **Isolation and protection**: one program can't read another's memory or crash the machine. The CPU itself enforces this with **privilege levels**: the OS **kernel** runs in *kernel mode* with full access; your apps run in *user mode* and must ask the kernel for anything sensitive.
3. **Abstraction**: instead of disk sectors you get files; instead of network card registers you get sockets; instead of one CPU you appear to have it all to yourself.

The **kernel** is the core that does this (Linux, the Windows NT kernel, XNU in macOS and iOS). Everything else (shells, desktops, apps) runs on top. Android runs on the Linux kernel; nearly all cloud servers run Linux.

5. Processes and threads

A process is a running program with its own private memory. A thread is one line of execution inside a process; threads share the process's memory.

	Process	Thread
Memory	Its own, isolated	Shared with other threads in the process
Creating one	Relatively expensive	Cheaper
If it crashes	Others survive	Can take down the whole process
Communication	Via the OS (pipes, sockets, files)	Just read and write shared variables (dangerous)
Example	Each Chrome tab is a separate process	One server process handling many requests on many threads

A process's memory has a standard layout: the **code**, **global data**, the **heap** (memory allocated as the program runs, e.g. lists and objects) and a **stack** per thread (local variables and the chain of function calls). A **stack overflow** is literally running out of stack, usually from runaway recursion; a **memory leak** is heap memory that's allocated and never freed, so the process grows until the OS kills it.

Each process has a **PID**, an owner, open files and an environment. Processes form a tree: your shell starts a program as a child. When you run `python app.py`, the OS creates a process, loads the interpreter's code, and starts its main thread.

Python note: CPython's **GIL** (global interpreter lock) lets only one thread run Python code at a time, so threads help with waiting on I/O but not with CPU-heavy work; for that you use multiple processes. (Newer Python versions are making the GIL optional; check the current state before relying on either behaviour.)

6. Scheduling and context switching

With more runnable threads than CPU cores, the OS slices time between them so fast that they all appear to run at once.

- Each thread is **running**, **ready** (waiting for a CPU) or **blocked** (waiting for something: disk, network, a lock, a timer).
- The **scheduler** picks which ready thread runs next on each core, for a short **time slice** (a few milliseconds), then may switch.
- A **context switch** saves the current thread's registers and program counter and loads another's. It costs microseconds, plus colder caches afterwards. Thousands per second is normal; millions is a problem.

Scheduling goals pull against each other:

Goal	Who cares
Throughput: finish as much work as possible	Batch jobs, data pipelines
Latency/responsiveness: react quickly to input	Phones, desktops, web servers
Fairness: nobody starves	Shared servers
Deadlines: guaranteed timing	Real-time systems: car brakes, audio, industrial control

The textbook algorithms (first-come-first-served, shortest-job-first, round robin, priority queues, multilevel feedback queues) are different trade-offs among these. Linux's default scheduler aims for fairness: each thread gets a share of CPU proportional to its priority.

Why this matters day to day: a server with 4 cores and 400 busy threads spends real time switching instead of working. A CPU-heavy task in a web server starves request handling. And "load average" in `top` is roughly how many threads want a CPU right now; above the core count, things queue.

7. Virtual memory

Every process believes it has a huge, private, contiguous memory starting at zero. The OS and CPU translate those fake addresses into real RAM behind the scenes.

- Memory is split into fixed-size **pages** (usually 4 KB). Each process has a **page table** mapping its virtual pages to physical frames in RAM. The CPU's **MMU** does the translation on every memory access, with a small cache of recent translations (the TLB).
- **Isolation for free**: process A's page table simply has no entries pointing at process B's memory, so A can't touch it. A bad pointer gives a **segmentation fault**, not someone else's data.
- **Lazy allocation**: pages are only given real RAM when first touched. A program can reserve 10 GB and use 100 MB.
- **Sharing**: two processes running the same program can map the same read-only code pages, saving RAM. Libraries are shared this way.
- **Swapping / paging out**: when RAM is full, the OS moves rarely used pages to disk. Touching one again triggers a **page fault**, and the OS reads it back. If a machine is constantly doing this (**thrashing**), everything slows to a crawl.
- **Out of memory**: on Linux, if there's truly nothing left, the **OOM killer** picks a process (usually the biggest) and kills it. "My container keeps restarting with exit code 137" is almost always this.
- **Memory-mapped files**: a file can be mapped into memory and read like an array; the OS loads pages on demand. Databases and model-loading libraries use this heavily.

> **In one sentence for a non-technical listener:** each program gets its own imaginary notebook with numbered pages, and the OS keeps a private index of which real shelf each page is actually on, moving rarely used pages to the basement when the shelves fill up.

8. Concurrency

When things happen at the same time and share data, the order of operations becomes unpredictable, and bugs appear that only show up sometimes.

Race conditions

`count += 1` looks atomic but is really three steps: read `count`, add 1, write it back. Two threads interleaving those steps can both read 5 and both write 6, losing an increment. This is a **race condition**: the result depends on timing. The same bug at business scale: two requests both check "seats left > 0", both see 1, both book the last seat.

Locks and their costs

- A **mutex** (lock) lets only one thread into a **critical section** at a time. Fixes races, but threads now wait.
- **Deadlock**: thread 1 holds lock A and waits for B; thread 2 holds B and waits for A. Both wait forever. The four conditions (mutual exclusion, hold-and-wait, no preemption, circular wait) all have to hold; the standard fix is to always acquire locks in the same global order.
- **Starvation**: a thread never gets the lock because others keep winning.
- **Atomic operations** (compare-and-swap) do tiny updates safely without a full lock. Databases offer the same idea at a higher level: transactions, row locks, and `UPDATE seats SET left = left - 1 WHERE id = 7 AND left > 0`, which checks and changes in one atomic step (Part 4).

Async vs threads

Most server work is *waiting*: for the database, an LLM API, a file upload. Two ways to wait on many things at once:

	Threads	Async (event loop)
Model	One thread per task; the OS switches between them	One thread; tasks voluntarily yield at each `await`
Good for	Mixed work, libraries that block	Thousands of concurrent network waits (chat, streaming, API fan-out)
Main risk	Races on shared data, memory per thread	One slow CPU-heavy task freezes everything
Examples	Java servers, Python `threading`	Node.js, Python `asyncio`, FastAPI

Async doesn't make work faster; it makes *waiting* cheaper. An agent that calls five tools in parallel with `asyncio.gather` finishes in the time of the slowest call instead of the sum.

9. Files, I/O and system calls

Programs don't touch hardware. They ask the kernel through system calls, and the kernel does it for them.

- A **system call** is the doorway from user mode into the kernel: `open`, `read`, `write`, `close`, `socket`, `fork`, `exec`, `mmap`. Every file read, network send and process launch goes through one. `strace` on Linux shows them live.
- A **file descriptor** is a small number the kernel hands back for each open file, socket or pipe. "Too many open files" means a process hit its descriptor limit, usually because connections or files are being opened and never closed.
- **File systems** (ext4, APFS, NTFS) organise disk blocks into files and directories, track metadata (owner, permissions, timestamps) in **inodes**, and use a **journal** so a crash mid-write doesn't corrupt the structure.
- **Buffering and the page cache**: writes usually land in RAM first and reach disk later. A program that must not lose data calls `fsync`. Databases do this on every commit, which is why commits cost disk-latency time.
- **Permissions**: on Unix, every file has read/write/execute bits for owner, group and others (`chmod 600 secrets.env`). Running a service as root means a bug can do anything; least privilege (Part 6) starts here.
- **Everything is a file** (Unix philosophy): devices, pipes and even process information (`/proc`) look like files, so the same tools work on all of them.
- **Interrupts**: hardware signals the CPU that something happened (a packet arrived, a key was pressed), so the OS doesn't have to keep checking. **DMA** lets devices copy data into memory without the CPU doing it byte by byte.

10. Virtual machines and containers

Both let you run many isolated workloads on one machine. They isolate at different levels, and that difference explains their cost and speed.

	Virtual machine	Container
What's isolated	A whole computer, including its own kernel	A group of processes sharing the host's kernel
How	A **hypervisor** (KVM, VMware, Hyper-V) emulates hardware	Kernel features: **namespaces** (own view of processes, network, files) and **cgroups** (limits on CPU, memory)
Start time	Tens of seconds to minutes	Under a second
Size	GBs (a full OS)	MBs (just the app and its libraries)
Isolation strength	Strong: separate kernels	Weaker: one kernel bug can affect all containers
Typical use	Cloud servers (an EC2 instance is a VM), running Windows on a Mac	Packaging and deploying apps; Kubernetes

A **container image** (built from a `Dockerfile`) bundles the app with exactly the libraries it needs, so "works on my machine" becomes "works everywhere". A running container is just ordinary Linux processes that the kernel shows a restricted view of the system to, and holds to memory and CPU limits. Exceeding the memory limit gets the container killed (exit 137 again).

In the cloud these stack: your container runs inside a VM, which runs on a physical server shared with other customers. Serverless platforms use lightweight micro-VMs (like AWS Firecracker) to get VM-strength isolation with near-container start times.

11. Labs

Lab 1. Meet your processes. Open Task Manager (Windows), Activity Monitor (Mac) or `top` / `htop` (Linux/WSL). Find: the process using the most memory, how many threads Chrome has, the number of CPU cores, and the load average or CPU graph. Open 20 browser tabs and watch memory.

Lab 2. Feel the memory hierarchy. Run this and compare times:

```
import time, random
N = 5_000_000
data = list(range(N))
order = list(range(N)); random.shuffle(order)
t = time.perf_counter(); s = 0
for i in range(N): s += data[i]
print("sequential", round(time.perf_counter() - t, 2))
t = time.perf_counter(); s = 0
for i in order: s += data[i]
print("random    ", round(time.perf_counter() - t, 2))
```

On a test run, random order took about ten times as long on the same data. The work is identical; the difference is mostly cache misses.

Lab 3. Cause a race condition, then fix it.

```
import threading, time
count = 0
def work():
    global count
    for _ in range(10_000):
        tmp = count      # read
        time.sleep(0)    # let another thread run here, as a real I/O wait would
        count = tmp + 1  # write
threads = [threading.Thread(target=work) for _ in range(4)]
[t.start() for t in threads]; [t.join() for t in threads]
print(count, "expected", 40_000)
```

The total comes out far short (around 10,000), because threads keep overwriting each other's updates. The `sleep(0)` stands in for any pause between the read and the write, like a database call; without it the race still exists but shows up only rarely, which is exactly what makes these bugs hard to catch. Now add `lock = threading.Lock()` at the top and put the three lines inside `with lock:`. It comes out exactly 40,000.

Lab 4. Async vs sequential waiting.

```
import asyncio, time
async def fake_api(i):
    await asyncio.sleep(1)   # stands in for a network call
    return i
async def main():
    t = time.perf_counter()
    for i in range(5): await fake_api(i)
    print("one by one", round(time.perf_counter() - t, 1))
    t = time.perf_counter()
    await asyncio.gather(*(fake_api(i) for i in range(5)))
    print("together  ", round(time.perf_counter() - t, 1))
asyncio.run(main())
```

Lab 5. Look inside a container. With Docker installed: `docker run --rm -it --memory=100m python:3.12-slim bash`. Inside, run `ps aux` (only a couple of processes), `hostname` (not your machine's) and `cat /proc/1/cgroup`. Then try `python -c "x = bytearray(300_000_000)"` and see it get killed by the memory limit.

Lab 6 (optional, fun). Play the first few levels of *Turing Complete* or do the first two projects of *Nand2Tetris*: build NOT, AND, OR and an adder from NAND gates.

12. Self-check questions

1. **What does a CPU do, in one sentence?** It repeatedly fetches an instruction from memory, decodes it, executes it on registers using the ALU, and moves to the next one, billions of times a second.
2. **Why do AI workloads run on GPUs?** Neural networks are mostly huge matrix multiplications: the same operation on lots of numbers. GPUs have thousands of simple cores built for exactly that.
3. **Why does iterating an array in order beat random access?** Caches load 64-byte lines and prefetch; sequential access reuses what's already loaded, random access keeps missing to slow RAM.
4. **Process vs thread?** A process has its own isolated memory; threads inside one process share memory. Threads are cheaper to create but can corrupt each other's data and crash together.
5. **What is a context switch and why can too many hurt?** Saving one thread's state and loading another's. Each costs time and cools caches; with far more busy threads than cores, the machine spends a lot of time switching.
6. **What problem does virtual memory solve?** It gives each process a private address space (isolation), lets memory be allocated lazily and shared, and lets the OS move pages to disk when RAM is short.
7. **Your container restarts with exit code 137. First guess?** It exceeded its memory limit and was killed (OOM). Check memory usage and limits, look for leaks.
8. **Explain a race condition with a non-code example.** Two clerks both check the ledger, see one seat left, and both sell it. The check and the sale needed to be one indivisible step.
9. **How do you prevent deadlock in practice?** Always take locks in the same order, hold them briefly, use timeouts, or avoid shared locks (let the database do atomic updates).
10. **When is async better than threads?** When the work is mostly waiting on many network calls at once (chat servers, API fan-out, streaming). Not for CPU-heavy work.
11. **What is a system call?** A request from a program to the kernel to do something privileged: open a file, send on a socket, start a process.
12. **Why call fsync?** Writes sit in RAM first; fsync forces them to disk so they survive a crash or power cut.
13. **Container vs VM, in one line each?** A VM is a whole simulated computer with its own kernel. A container is a group of processes sharing the host's kernel, with a restricted view and resource limits.
14. **Why is a container less isolated than a VM?** All containers share one kernel, so a kernel vulnerability can cross between them; VMs have separate kernels behind a hypervisor.

13. Resources

Check these are still available before committing time.

- **Crash Course Computer Science** (YouTube, episodes 1–20). The gentlest possible on-ramp from transistors to operating systems.
- **Ben Eater's 8-bit breadboard computer** (YouTube). A real CPU built from chips, explained wire by wire.
- **Nand2Tetris** (nand2tetris.org, free; also a Coursera course). Build a computer from NAND gates up; the first half (hardware) covers this chapter.
- **Remzi and Andrea Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*** (free at ostep.org). Short, witty chapters on virtualisation, concurrency and persistence. Read selectively: processes, scheduling intro, address spaces, paging, locks, condition variables, file system intro.
- **Julia Evans's zines** on Linux, processes and containers (wizardzines.com).
- **Turing Complete** (game) for the gates-to-CPU section.